Advanced Terraform on AWS Lab 2 Hygiene and Validation

Expected Duration 1
Teaching Points 1
Before You Begin 1
Lab rules 3
Phase 0 – Chaos 3
Phase 1 – Structure 6
Phase 2 – Determinism 10
Phase 3 – Visibility 15
Phase 4 – Enforcement 22
Lab Conclusions 25

Expected Duration

This lab should take 60-70 minutes to complete

Avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve will be lost and the lab learning value will therefore be reduced.

Teaching Points

By the end of this lab, you will be able to:

- Recognize the risks of poor hygiene and validation in Terraform configurations.
- Understand how inconsistent keys can cause configuration churn.
- Use locals to enforce naming conventions and tag consistency.
- Apply string and collection functions to normalize inputs.
- Implement guardrails using variable validation and lifecycle conditions.
- Use terraform console to safely inspect and test expressions.
- Produce deterministic, predictable Terraform configurations.

Before You Begin

The lab2 folder used for this lab contains two sub-directories; '**guided**' and '**phases**'.

All tasks are performed in the **guided** directory, where you have access to a script named '**phases.cmd.**' Running this 'Phase switcher' script allows you to switch between lab phases, as outlined in the instructions which follow. Selecting a phase will copy new files into the guided folder, simulating progressive modifications to the original 'chaotic' files. The providers.tf file which exists in the guided folder initially will remain static throughout the lab…

```
========================================
  Terraform Lab 2 - Phase Switcher
========================================
 0 Chaos (Unpredictable behaviour)
 1 Structure (Readable configuration)
 2 Determinism (Stable, predictable outputs)
 3 Visibility (Detecting problems)
 4 Enforcement (Preventing problems)
 X Exit

Select phase to load:
```

Important: Do not have any **guided** files active in VSC editor as you switch between phases. Doing so can cause 'On-disk vs In-memory' file version conflicts and may lead to 'phantom' error displays. If any of these appear then simply close and re-open VSC to clear them. All the supplied code has been extensively evaluated and works 'as-is'

You can use the 'compare' feature in VSC to view the file differences as you move through this lab.

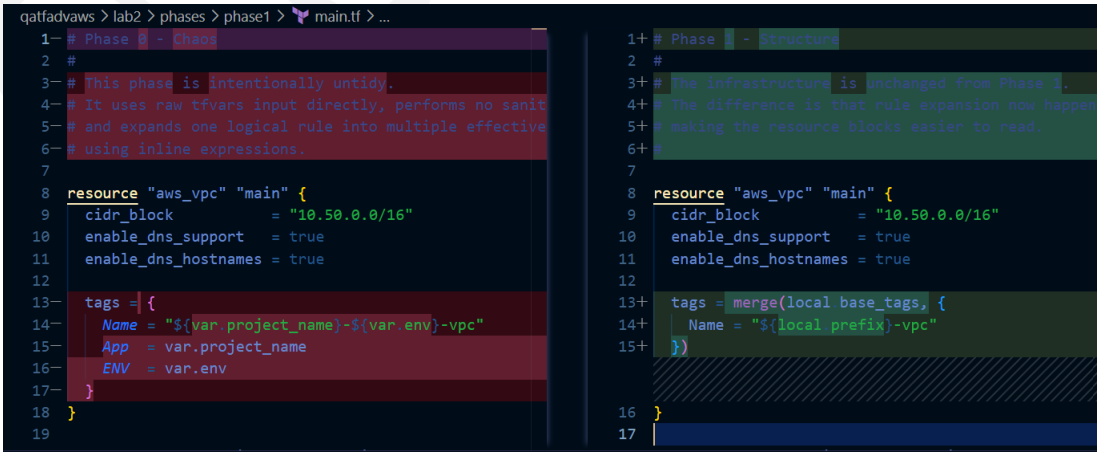
Try this now; to compare **main.tf** changes from Phase 0 to Phase 1 ..

In explorer, navigate to Phases/Phase0 and right click **main.tf**

Select "Select for compare"

Navigate to Phases/Phase1 and right-click **main.tf**

Select "Compare with selected"



Lab rules

In production projects, tfvars files are often owned or populated by another team or by a pipeline. Your Terraform code must therefore sanitize provided input, normalize it and enforce guardrails without assuming the input is perfect or that you can change it.

Phase 0 – Chaos

1. Move into **lab2\guided**
2. Run **phases.cmd** and select **Phase 0**.
3. Review the provided files to gain an understanding of the original code and the intended resource provisioning. This understanding is *critically* important in order to appreciate the changes you will make.

   Understanding the Starting Point

   The intent of this code is to create;

   A VPC named **landing-zone-dev-vpc**

   A Security Group named **landing-zone-dev-sg**

   Security Group ingress rules **allow_http_to_office**

   Security Group ingress rules **allow_https_to_office**

   Security Group egress rules **allow_http_to_partner**

   This lab does **not** start from a clean or well-structured configuration.

   Instead, it deliberately uses **chaotic, inconsistent input data** to simulate a more realistic scenario.

   What are we trying to achieve?

   In this phase, you will:

   - observe how Terraform behaves with untidy input
   - see how that input is expanded into multiple AWS resources
   - identify where errors occur and where they do not

   The goal is not to permanently fix anything yet.

   The goal is to **observe and understand behaviour**.

   Why is the data chaotic ?

   The input data in terraform.tfvars has been intentionally designed to include common real-world issues:

   - CIDR values with leading and/or trailing whitespace
   - duplicate CIDR entries
   - inconsistent casing (TCP, Tcp, tcp)

- inconsistent rule names
- repeated or overlapping values

These issues are typical in environments where:

- multiple teams contribute configuration
- standards are not enforced
- data is copied and reused

What will Terraform do with this data?

Terraform will:

- read the input variables
- expand each logical rule into multiple effective rules
- attempt to build the final resource configuration

At this stage, correctness is enforced by the AWS provider, not Terraform itself. Terraform evaluates and passes data, but does not guarantee that it is valid or safe.

Why might errors occur?

Not all problems are treated the same.

Some issues are rejected immediately

AWS performs validation on certain inputs, particularly CIDR values.

This means values such as:

- " 10.0.1.0/24 " (whitespaces)
- "10.0.0.0/33" (invalid range)

may cause errors during planning and apply.

These errors occur because the provider expects a valid IPv4 network format.

Other issues are not rejected by Terraform

Some values are technically valid, even if they are undesirable.

These include:

- duplicate CIDRs
- overlapping CIDRs
- inconsistent naming or casing

Some will pass through Terraform and AWS without error, others will pass through Terraform only to be rejected by AWS.

What should you expect in this phase?

In this phase of the lab, some of the input is *invalid* enough to prevent a successful deployment from taking place at all. In particular, malformed CIDR values will be rejected during plan/apply before AWS creates any resources.

This is expected. The purpose of this phase is to show that Terraform can begin interpreting untidy input, but that badly formed values can prevent the deployment from succeeding. The instability is deliberate.

4. Focus on ' **terraform.tfvars** ...
    - The http/https allowlists contain duplicates and whitespace
    - Security Group rule keys are in mixed styles (allow_http_from_office, Allow-HTTPS-From-Office)
    - Security Group rule **source_cidrs** contains duplicates and whitespace
    - Security Group rule **allow_group** uses different casing (HTTP vs https).

5. Plan this configuration...

    ```
    terraform init
    terraform plan
    ```

    Errors are displayed relating to erroneous cidrs

6. Attempt to apply this configuration

    ```
    terraform apply --auto-approve
    ```

    Note that the attempt fails due to the severity of the initial errors relating to malformed cidrs

    This baseline code highlights the need for strong configuration control.

Manual fixes

The degree of 'messiness' surrounding input data and its impact varies. Next you will reduce the 'messiness' manually to highlight this. **Note:** Moving forward, you will reinstate the original file structure and deal with the mess programmatically as would be the case in a production environment.

7. In **terraform.tfvars**, remove all leading and trailing whitespaces present for cidrs, there are four entries that need updating on lines 16, 21, 22 and 32.

8. Plan this configuration ...

    ```
    terraform plan
    ```

    Planning should be successful. Revisit the CIDRs and ensure **all** whitespacing has been removed if you receive errors at this point.

9. Apply this configuration

```
terraform apply --auto-approve
```

10. This results in a deployment with errors, the anti-pattern of what we want. View the deployment in the Portal along with the error messages relating to duplicate cidrs
11. In **terraform.tfvars**, remove the duplicate cidrs (comment lines 15 and 23, and remove duplicate on line 41)
12. Destroy current resources and then apply this configuration…

```
terraform destroy --auto-approve
terraform apply --auto-approve
```

13. This produces no errors, but the descriptions given to the ingress and egress rules are not standardized …

Source	Description
10.0.0.0/24	allow_http_from_office
10.0.12.0/24	allow_http_from_office
10.0.1.0/24	allow_http_from_office
10.0.2.0/24	allow_http_from_office
10.0.3.0/24	Allow-HTTPS-From-Office
10.0.12.0/24	Allow-HTTPS-From-Office
10.0.1.0/24	Allow-HTTPS-From-Office

14. A more insidious problem is the index ordinality being used in main.tf when provisioning the ingress and egress rule. To highlight this, add a second cidr “10.0.11.0/24” ***before*** the current cidr on line 41 of **terraform.tfvars**…

```
37    Allow-HTTPS-Office = {
38      type              = "INGRESS"
39      protocol          = "Tcp"
40      destination_ports = ["443"]
41      source_cidrs      = ["10.0.11.0/24","10.0.10.0/24"]
42      allow_groups      = ["internal_networks", "partner_networks"]
43      description       = "Allow-HTTPS-to-Office"
44    }
```

15. Plan this configuration change and note the proposed churn…

```
terraform plan
```

```
Plan: 2 to add, 9 to change, 1 to destroy.
```

16. Destroy all existing resources ahead of moving to the next phase…

```
terraform destroy --auto-approve
```

In Phase 1, the manual changes you have made are rolled back ahead of a phased process of cleansing, validating, and automating.

Phase 1 – Structure

In this phase, you will begin restructuring the configuration by introducing locals.

In Phase 0, Terraform was able to build the security group rules, but the logic for doing so was embedded directly inside the resource blocks...

```
32  resource "aws_vpc_security_group_ingress_rule" "rule" {
33    for_each = {
34  >   for idx, item in flatten([ ···
53    ]) : "${item.rule_key}|${item.port}|${idx}" => item
54    if lower(item.type) == "ingress"
55  }
```

This makes the configuration difficult to read, hard to debug, and awkward to extend. In this phase we refactor that expansion logic into locals. The input remains chaotic, and we are not yet trying to standardize it. The goal is simply to separate rule construction from resource declaration.

What are we trying to do?

Phase 1 is about:

- separating transformation from resource declaration
- making the rule-building pipeline visible
- keeping the original input intact

It is **not** yet about:

- trimming whitespace
- lowercasing values
- removing duplicates
- validating CIDRs
- rejecting bad input

1. Close any open files in VSC.
2. Run the phase switcher and select **Phase 1**
3. A new file **locals.tf** is introduced whilst **main.tf** and **outputs.tf** are updated.
4. Review **locals.tf** You will see a number of new local values being defined.

   Environment locals

   ```
   prefix = "${var.project_name}-${var.env}"

   base_tags = {
     app = var.project_name
     env = var.env
   }
   ```

   These locals can now be referenced by resource blocks rather than each resource block having to do the concatenation inline. Note that there is no sanitization of the raw variables as yet. This will come in Phase 2.

   Rules locals

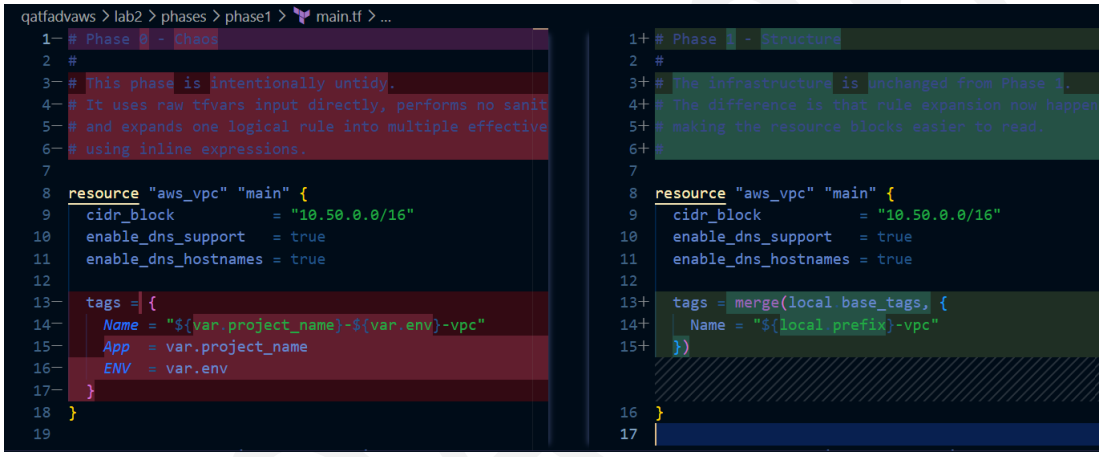
© QA 2026

```
raw_ingress_rules = {
  for idx, item in flatten([
    for rule_key, rule in var.security_group_rules : [
      for port in rule.destination_ports : [
        for cidr in concat(
```

```
raw_egress_rules = {
  for idx, item in flatten([
    for rule_key, rule in var.security_group_rules : [
      for port in rule.destination_ports : [
        for cidr in concat(
```

These locals remove the complexity of *deriving* the actual rules away from the resource blocks that will consume them, making the resource provisioning code simpler to understand and debug.

5. Review the changes in **main.tf**



Before (Phase 0):

- resource names built directly from raw variables
- repeated string concatenation
- repeated tag definitions
- complex rule extraction from raw variables

After (Phase 1):

- resource names use local.prefix
- tags use local.base_tags
- repetition has been removed
- The rules resource blocks now reference the rules locals, making them simpler to understand and troubleshoot

Important observation:

The security group rule logic has not changed.

- CIDR values are still messy
- duplicate values still exist
- whitespace is still present

- rule keys are still inconsistent
- keys based on index ordinality still exist.

6. Run planning:

   ```
   terraform plan
   ```

   What should you expect?

   - Repetitive resource naming will now be consistent
   - Tags will now be consistent across all resources
   - The configuration is structurally improved

   The configuration is now structurally improved and easier to understand. However, the plan may still fail due to unresolved data quality issues. This is expected.

   Why it might still fail:

   - whitespace in CIDR values is still present
   - duplicate values still exist
   - invalid values are still passed to the provider

7. Terraform inspection:

   ```
   terraform console
   local.prefix
   local.base_tags
   ```

```
PS C:\qatfadvaws\lab2\guided> terraform console
> local.prefix
"landing-zone-dev"
> local.base_tags
{
  "app" = "landing-zone"
  "env" = "dev"
}
> ^C
```

```
local.raw_ingress_rules
```

```
> local.raw_ingress_rules
{
  "Allow-HTTPS-Office|443|0" = {
    "cidr_ipv4" = "10.0.10.0/24"
    "description" = "Allow-HTTPS-to-Office"
    "port" = "443"
    "protocol" = "Tcp"
    "rule_key" = "Allow-HTTPS-Office"
    "type" = "INGRESS"
  }
  "Allow-HTTPS-Office|443|1" = {
    "cidr_ipv4" = "10.0.10.0/24"
    "description" = "Allow-HTTPS-to-Office"
    "port" = "443"
    "protocol" = "Tcp"
    "rule_key" = "Allow-HTTPS-Office"
    "type" = "INGRESS"
  }
  "Allow-HTTPS-Office|443|2" = {
```

```
local.raw_egress_rules
```

```
> local.raw_egress_rules
{
  "Allow-HTTPS-Partner|443|10" = {
    "cidr_ipv4" = "172.16.1.0/24"
    "description" = "Allow-HTTPS-to-Partner"
    "port" = "443"
    "protocol" = "Tcp"
    "rule_key" = "Allow-HTTPS-Partner"
    "type" = "EGRESS"
  }
  "Allow-HTTPS-Partner|443|7" = {
    "cidr_ipv4" = "172.16.0.50/32"
    "description" = "Allow-HTTPS-to-Partner"
    "port" = "443"
    "protocol" = "Tcp"
    "rule_key" = "Allow-HTTPS-Partner"
    "type" = "EGRESS"
  }
```

Sidebar: Why do the indexes jump?

The number at the end of each key is the original `idx` value from the fully expanded rule list. Terraform first expands **all** rules - ingress and egress - into one combined list. Only after that does it filter them into `raw_ingress_rules` and `raw_egress_rules`.

That means the index is shared across both collections. So if entries `7`, `8`, `9` and `10` are egress rules, they will not appear in `local.raw_ingress_rules`. This is why the ingress output may appear to jump from `6` to `11`. The reverse is also true: `local.raw_egress_rules` only shows the egress entries, but those entries still keep their original indexes from the combined list.

This is **ordinality**: the resource identity is partly based on the item's position in a list.

Why this matters

As you learnt in Lab1, index-based keys are fragile. As demonstrated in Phase 0, if a new CIDR is inserted earlier in the input, later indexes may change even though the actual rules have not changed. Terraform can then interpret the change as a destroy/recreate operation.

Later in the lab, this is fixed by replacing index-based identity with meaning-based identity.

8. **Key point**: The configuration is better structured, but the data is still unreliable.
9. **What comes next**: Phase 2 will begin cleaning and normalizing the data.

Phase 2 – Determinism

What are we trying to achieve?

In Phase 1, the configuration was restructured to improve readability and maintainability. The rule expansion logic was moved into locals, making the configuration easier to understand and debug.

However, the underlying issue remains: The input data is still chaotic and unreliable.

In this phase, you will transform that raw input into a clean, consistent, and deterministic internal model.

The **terraform.tfvars** file remains intentionally chaotic:

- CIDRs contain whitespace
- duplicate values exist
- casing is inconsistent
- rule keys and descriptions vary in format

In a real-world scenario, you often cannot control this input. Instead, your Terraform configuration must adapt to chaotic input and produce clean, predictable output

What is normalization?

Normalization is the process of converting inconsistent data into a standard, canonical form. In this phase, normalization includes:

- trimming whitespace
- converting values to consistent casing
- removing duplicate entries
- standardizing naming conventions
- resolving allow groups into concrete CIDR values
- generating stable rule identities

Why is this important?

In Phase 1:

- rule identity depended on index position
- duplicate values created duplicate rules
- whitespace and casing caused inconsistencies
- small ordering changes caused configuration churn

This made the configuration:

- unpredictable
- fragile
- difficult to maintain

In Phase 2, we fix that. Rule identity will now be based on meaning, not position.

What will change in this phase?

In this phase, you will:

- introduce canonical naming for project and environment
- clean and normalize allow group definitions
- clean and normalize rule definitions
- resolve allow groups into concrete CIDR lists
- remove duplicates and blank values
- standardize descriptions
- construct stable rule keys
- eliminate index-based identity

What will NOT change?

The following remain unchanged:

- the terraform.tfvars file is not modified
- no validation or rejection of bad input is performed
- Terraform may still pass invalid values to the provider

This is intentional. We are cleaning the data, not enforcing correctness.

1. Ensure no files are open in the guided directory.
2. Run the phase switcher and select **Phase 2**

3. Review **locals.tf**

   This file now performs all transformation logic.

   Environment normalization

   You will see:

   env_canon
   project_canon
   prefix

   These locals:

   - remove whitespace
   - enforce lowercase
   - strip invalid characters
   - produce consistent naming

   This ensures all resource names and tags are consistent and predictable

   Allow group normalization

   allow_groups_clean

   This local:

   - trims whitespace from CIDRs
   - removes blank values
   - removes duplicates
   - standardizes allow group names

   Rule normalization

   security_group_rules_clean

   Each rule is cleaned by:

   - normalizing rule keys
   - converting type and protocol to lowercase
   - trimming and deduplicating ports
   - trimming and deduplicating CIDRs
   - standardizing allow group references
   - normalizing descriptions

   Descriptions are now:

   - lowercase
   - consistently formatted
   - deterministic

   Rule resolution

   ingress_rules_resolved
   egress_rules_resolved

©QA 2026

These locals:

- combine rule-specific CIDRs with allow group CIDRs
- produce complete network sets for each rule

Directional behaviour is now explicit:

- ingress uses source_cidrs
- egress uses destination_cidrs

Effective rule expansion

effective_ingress_rules
effective_egress_rules

Each rule is expanded into:

- protocol
- port
- CIDR

Critical change: Rule identity

Phase 1 locals used: rule_key | port | index (This is the ordinality problem)

Line 55 >>> "${item.rule_key}|${item.port}|${idx}" => item

Phase 2 locals now uses: rule_key | protocol | port | cidr

Line 146 >>> "${rule_key}|${rule.protocol}|${port}|${cidr}"

This means rule identity is now based on what the rule is, not where it appears.

4. Review **main.tf**

   You will observe:

   - resources now consume effective_*_rules
   - no inline transformations exist
   - no data cleaning occurs in resource blocks

   This is a key milestone: Resources now consume clean data, rather than fixing it.

5. Run planning

   ```
   terraform plan
   ```

   What should you expect?

   Compared to Phase 1:

   - duplicate rules are removed
   - whitespace no longer causes duplication
   - rule identity is stable
   - Terraform no longer proposes unnecessary replacements
   - configuration behaviour is predictable

6. Apply the deployment

   ```
   terraform apply --auto-approve
   ```

7. Confirm the new rule identities in state...

   ```
   terraform state list
   ```

   ```
   PS C:\qatfadvaws\lab2\guided> terraform state list
   aws_security_group.main
   aws_vpc.main
   aws_vpc_security_group_egress_rule.rule["allow-https-partner|tcp|443|172.16.0.50/32"]
   aws_vpc_security_group_egress_rule.rule["allow-https-partner|tcp|443|172.16.1.0/24"]
   aws_vpc_security_group_egress_rule.rule["allow-https-partner|tcp|443|172.16.2.0/24"]
   aws_vpc_security_group_ingress_rule.rule["allow-http-office|tcp|80|10.0.0.0/24"]
   aws_vpc_security_group_ingress_rule.rule["allow-http-office|tcp|80|10.0.1.0/24"]
   ```

8. Test that we have eliminated the churn due to the index ordinality used in Phase 0. In **terraform.tfvars**, re-add a second cidr "10.0.11.0/24" ***before*** the current cidrs on line 41 (leave the duplicated cidr, our code now deals with that)...

   ```
   37  Allow-HTTPS-Office = {
   38    type              = "INGRESS"
   39    protocol          = "Tcp"
   40    destination_ports = ["443"]
   41    source_cidrs      = ["10.0.11.0/24","10.0.10.0/24", "10.0.10.0/24"]
   42    allow_groups      = ["internal_networks", "partner_networks"]
   43    description       = "Allow-HTTPS-to-Office"
   44  }
   ```

9. Plan the deployment...

   ```
   terraform plan
   ```

   The plan is 1 to add, 0 to change, 0 to destroy. No churn !

   Remove the new cidr from line 41.

10. Run:

    ```
    terraform console
    ```

    Inspect:

    ```
    local.allow_groups_clean
    local.security_group_rules_clean
    local.effective_ingress_rules
    local.effective_egress_rules
    ```

    Observe:

    - cleaned CIDRs
    - consistent naming
    - removal of duplicates
    - stable rule structures

Checkpoint: What have we achieved?

We have:

- transformed chaotic input into a clean internal model
- removed duplicate and inconsistent values
- eliminated index-based identity
- made the configuration deterministic

Important observation

Even though the data is now clean:

- invalid CIDRs are still not explicitly rejected
- unknown allow groups are not enforced
- incorrect values may still reach the provider

Important distinction: Normalization makes the data consistent, but not necessarily correct. At this stage, invalid or unsafe values may still exist. We have improved structure and predictability, but we have not yet enforced correctness. This is expected.

Key takeaway

Phase 1 made the configuration readable
Phase 2 makes the configuration consistent

What comes next?

In the next phase, you will introduce checks into the code.

This will allow you to identify:

- invalid CIDRs
- erroneous protocols and ports
- incorrect allow group usage

Phase 3 – Visibility

What are we trying to achieve?

In earlier phases, some invalid inputs were caught by Terraform or the cloud provider. However, these errors were often late, unclear, or unrelated to the intent of the configuration.

In this phase, we reduce reliance on provider-side validation and introduce explicit checks at the Terraform layer.

Terraform will now identify potential issues and surface them clearly, but it will still allow deployment to proceed; we are introducing detection, not enforcement.

1. Ensure no files are open in the guided directory.
2. Run the phase switcher and select **Phase 3**

3. Review **locals.tf** New helper locals have been introduced. Each will be discussed in turn.
4. Review **main.tf**

   New check and assertions have been introduced that reference the helper locals. These are currently commented out and will be uncommented and discussed in turn
5. Verify there is no change to the current deployment;

   ```
   terraform plan
   ```

   This should result in a no-op, showing that Phase 3 does not make any changes to the current deployment.
6. Destroy all fifteen resources ahead of the testing that follows;

   ```
   terraform destroy --auto-approve
   ```

Check 1. Unknown allow group

Currently lines 86 and 100 of **locals.tf** silently tolerate missing groups:

lookup(local.allow_groups_clean, g, [])

That means a typo in **allow_groups** becomes an empty result, not an error. This is exactly the sort of thing that Checks should detect and catch.

7. Introduce a typo into **terraform.tfvars**;



   On line 42, remove the ‘s’ from ‘partner_networks’
8. Plan the deployment and note the new resource count

   ```
   terraform plan
   ```

   The plan should now be to deploy a total of thirteen resources.

   The missing resources are the two cidrs of the **partner_networks** allow group as these are no longer referenced by the **allow-https-office** ingress rules. This omission, due to a simple typo, could have major implications in a production environment.
9. Fix the typo in **terraform.tfvars** and introduce an unknown allow-group;

```
37  Allow-HTTPS-Office = {
38    type              = "INGRESS"
39    protocol          = "Tcp"
40    destination_ports = ["443"]
41    source_cidrs      = ["10.0.10.0/24", "10.0.10.0/24"]
42    allow_groups      = ["internal_networks", "partner_networks", "test_networks"]
43    description       = "Allow-HTTPS-to-Office"
44  }
```

On line 42, add the 's' back to form '**partner_networks'** and include a new, undefined allow_group '**test_networks'**

10. Plan the deployment and note the new resource count

    ```
    terraform plan
    ```

    The plan should now be to deploy the original fifteen resources.

    We are back to the original deployment count. The new, undefined allow_group '**test_networks,'** is simply treated as an empty result when looking it up in **allow_groups_cleaned**.

Both of these scenarios can be avoided by checking that the **allowed_groups** included in the rules *actually* exist in **allow_groups_cleaned**.

11. In **locals.tf** review lines 145-155;

```
145  unknown_allow_groups = {
146    for rule_key, rule in local.security_group_rules_clean :
147    rule_key => [
148      for g in rule.allow_groups : g
149      if !contains(keys(local.allow_groups_clean), g)
150    ]
151    if length([
152      for g in rule.allow_groups : g
153      if !contains(keys(local.allow_groups_clean), g)
154    ]) > 0
155  }
```

12. In **main.tf** uncomment lines 7-12;

```
7   check "unknown_allow_groups" {
8     assert {
9       condition     = length(local.unknown_allow_groups) == 0
10      error_message = "Unknown allow_groups were referenced. Every allow_group us
11    }
12  }
```

13. Plan the deployment and note that terraform flags the issue…

    ```
    terraform plan
    ```

```
Unknown allow_groups were referenced. Every allow_group used by a rule must exist.
Offending rules: {"allow-https-office":["test-networks"]}
```

14. Remove 'test-networks' from **terraform.tfvars** (line 42) and re-introduce the typo 'partner-network;'

```
37    Allow-HTTPS-Office = {
38      type              = "INGRESS"
39      protocol          = "Tcp"
40      destination_ports = ["443"]
41      source_cidrs      = ["10.0.10.0/24", "10.0.10.0/24"]
42      allow_groups      = ["internal_networks", "partner_network"]
43      description       = "Allow-HTTPS-to-Office"
44    }
```

15. Plan the deployment and note that terraform flags the issue...

    ```
    terraform plan
    ```

    ```
    Plan: 13 to add, 0 to change, 0 to destroy.
    ```

    ```
    Unknown allow_groups were referenced. Every allow_group used by a rule must exist.
    Offending rules: {"allow-https-office":["partner-network"]}
    ```

16. Verify that the remaining resources deploy...

    ```
    terraform apply --auto-approve
    ```

    ```
    Apply complete! Resources: 13 added, 0 changed, 0 destroyed.
    ```

17. Fix the typo in **terraform.tfvars**

18. Plan the deployment and note the results

    ```
    terraform plan
    ```

19. The plan should be to deploy the remaining resources now that all allow groups checks are passed.

20. Destroy all deployed resources ahead of the next checks...

    ```
    terraform destroy --auto-approve
    ```

Check 2. Empty rules

Rules with no effective CIDRs are a no-op. Currently there are no checks to flag this.

21. Test the current configuration by removing all cidrs used when defining the '**allow_http_office'** ingress rules in **terraform.tfvars** ...

```
27    security_group_rules = {
28      allow_http_office = {
29        type              = "INGRESS"
30        protocol          = "TCP"
31        destination_ports = ["80"]
32    #     source_cidrs      = [" 10.0.10.0/24 "]
33    #     allow_groups      = ["internal_networks"]
34        description       = "allow_http_to_office"
35      }
```

Comment lines 32 and 33 of **terraform.tfvars**

22. Plan the deployment..

```
terraform plan
```

The plan is for eleven resources to be added, not the original fifteen. The four *missing* resources are the **allow_http_office** ingress rules that originally included the three **internal_networks** cidrs and the one explicitly defined source cidr. This effectively leaves a no-op, no **allow_http_office** rules are created.

This highlights a critical behaviour: Terraform considers this configuration valid, even though it results in no effective rules. This is not a syntax error, and it is not rejected by the provider. It is logically incorrect, but structurally valid.

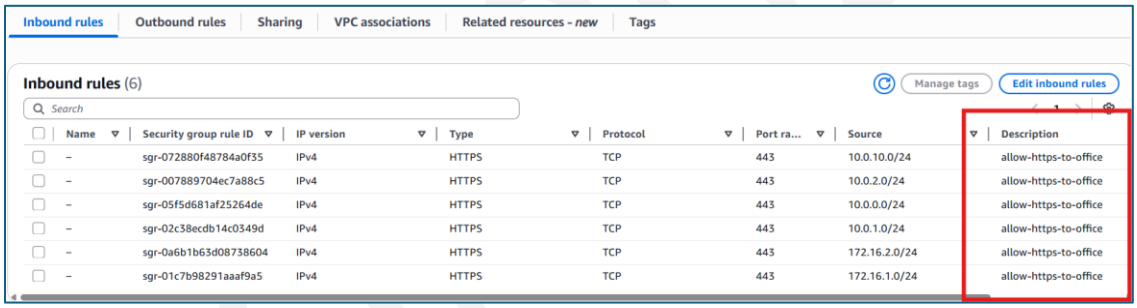
This is a key risk in infrastructure as code: Terraform will successfully apply configurations that are logically incorrect unless explicit guardrails are introduced.

This is why detection alone is not sufficient.

23. Apply the deployment and verify the no-op...

```
terraform apply --auto-approve
```

24. Switch to the Portal and verify the absence of any **allow_http_office** rules...



25. Destroy all deployed resources...

```
terraform destroy --auto-approve
```

26. Examine lines 157-167 of **locals.tf** ..

```
157  empty_ingress_rules = {
158    for rule_key, rule in local.ingress_rules_resolved :
159    rule_key => rule
160    if length(rule.resolved_cidrs) == 0
161  }
162
163  empty_egress_rules = {
164    for rule_key, rule in local.egress_rules_resolved :
165    rule_key => rule
166    if length(rule.resolved_cidrs) == 0
167  }
```

These workflow blocks examine the length of the sanitized ingress and egress cidrs, counting the number that are zero in length. A non-zero count indicates rules with no cidrs, and this can then be used as a check during the rules creation in **main.tf**

27. Review and then uncomment lines 14-26 in **main.tf** ...

```
14  check "non_empty_resolved_ingress_rules" {
15    assert {
16      condition     = length(local.empty_ingress_rules) == 0
17      error_message = "Every ingress rule must resolve to at least one effective CIDR.
18    }
19  }
20
21  check "non_empty_resolved_egress_rules" {
22    assert {
23      condition     = length(local.empty_egress_rules) == 0
24      error_message = "Every egress rule must resolve to at least one effective CIDR.
25    }
26  }
```

These assertions check the values of the **empty_ingress_rules** and **empty_egress_rules** locals and returns an error message if either/both are not zero, thus flagging no-ops.

28. Plan the deployment..

```
terraform plan
```

```
Warning: Check block assertion failed

  on main.tf line 16, in check "non_empty_resolved_ingress_rules":
  16:     condition     = length(local.empty_ingress_rules) == 0

    local.empty_ingress_rules is object with 1 attribute "allow-http-office"

Every ingress rule must resolve to at least one effective CIDR. Empty ingress rules: ["allow-http-office"]
```

29. Verify that the failed check does not halt the deployment of other resources...

```
terraform apply --auto-approve
```

Eleven resources should be deployed.

30. Uncomment lines 32 and 33 of **terraform.tfvars** and re-deploy...

```
terraform apply --auto-approve
```

The remaining four resources are now deployed.

31. Destroy all resources ahead of the next check...

```
terraform destroy --auto-approve
```

Check 3. Invalid Protocols

Rules with protocols other than "tcp", "udp", "icmp", or "-1" will generate errors. Currently there are no terraform checks to flag this and errors will only be caught late by the provider.

32. Update line 30 of **terraform.tfvars** to introduce an invalid protocol...

```
27  security_group_rules = {
28    allow_http_office = {
29      type              = "INGRESS"
30      protocol          = "T CP"
31      destination_ports = ["80"]
32      source_cidrs      = [" 10.0.10.0/24 "]
33      allow_groups      = ["internal_networks"]
34      description       = "allow_http_to_office"
35    }
```

Introduce a space between T and CP to form an invalid protocol in line 30.

33. Plan the deployment and verify that no errors are flagged...

```
terraform plan
```

```
Plan: 15 to add, 0 to change, 0 to destroy.
```

34. Verify that the deployment partially fails late...

```
terraform apply --auto-approve
```

```
Error: creating VPC Security Group Rule

  with aws_vpc_security_group_ingress_rule.rule["allow-http-office|t cp|80|10.0.1.0/24"],
  on main.tf line 76, in resource "aws_vpc_security_group_ingress_rule" "rule":
  76: resource "aws_vpc_security_group_ingress_rule" "rule" {

operation error EC2: AuthorizeSecurityGroupIngress, https response error StatusCode: 400,
Invalid value 't cp' for IP protocol. Unknown protocol.
```

Four errors occur related to the attempts to create the **allow-http-office** rules based on invalid protocols. The remaining eleven resources are deployed.

35. Review lines 169-176 of locals.tf...

```
169   allowed_protocols = toset(["tcp", "udp", "icmp", "-1"])
170
171   protocol_violations = {
172     for rule_key, rule in local.security_group_rules_clean :
173     rule_key => rule.protocol
174     if !contains(local.allowed_protocols, rule.protocol)
175   }
176 }
```

A local, **allowed_protocols**, contains all valid protocols. Another local, **protocol_violations**, counts the number protocols proposed in each rule, which are *not* in the **allowed_protocols** set. A number other than zero indicates an invalid protocol.

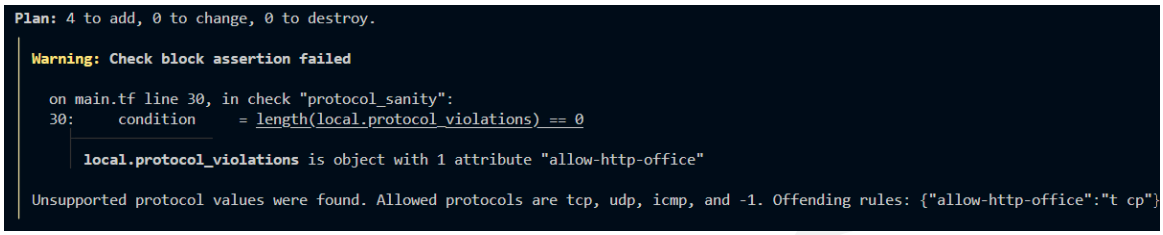
36. Uncomment lines 28-33 in **main.tf**...

```
28  check "protocol_sanity" {
29    assert {
30      condition     = length(local.protocol_violations) == 0
31      error_message = "Unsupported protocol values were found. Allowed pr
32    }
33  }
```

These assertions check the value of the **protocol_violations** locals and returns an error message if either/both are not zero.

37. Plan the deployment and verify that protocol errors are flagged…

```
terraform plan
```



38. Fix the typo in **terraform.tfvars** and apply the changes…

```
terraform apply --auto-approve
```

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

The *missing* four resources are now deployed.

Phase 4 – Enforcement

In earlier phases, the AWS provider enforced correctness.

In Phase 3, Terraform could detect problems, but still allowed deployment to proceed.

In this phase, Terraform takes ownership of enforcement, preventing invalid configurations from reaching the provider.

1. Ensure no files are open in the guided directory.
2. Run the phase switcher and select **Phase 4**
3. Review **locals.tf** ; No new helper locals have been introduced. The existing helper locals, used for checks in Phase 3 will be re-used in this phase for Guardrailing.
4. Review **main.tf** ; New lifecycle preconditions have been introduced that reference the helper locals.
5. Review **variables.tf** ; Validation rules are introduced for all defined variables.
6. Verify that these file changes does not affect the current deployment;

   ```
   terraform plan
   ```

   This should result in a no-op, showing that Phase 4 does not make any changes to the current deployment.

7. Destroy all current resources ahead of testing that follows;

   ```
   terraform destroy --auto-approve
   ```

The two enforcement layers

In this phase, we introduce two enforcement mechanisms: variable validation and lifecycle preconditions, each operating at a different stage of the Terraform workflow.

Variable validation checks incoming input values and fails early if they are unacceptable, preventing bad data from entering the configuration. It is best suited to simple, raw input rules.

Lifecycle preconditions operate later, evaluating the final, computed model after all transformations and normalization have taken place. This allows you to enforce rules based on what Terraform is actually about to build. In simple terms, *validation protects the front door, while preconditions halt Terraform before resource creation if the final computed model is invalid*

Mechanism	Used for	Behaviour	Pros	Cons
Variable validation	Raw input rules	Fails before planning continues	Fails early Clear error messages Easy for users to fix Keeps bad data out completely	Limited context (no dependency awareness) Can become overly restrictive Hard to express complex logic
Lifecycle preconditions	Derived/normalized rule outcomes	Fails before resource creation	Full visibility of computed data Can enforce real correctness Perfect for derived logic Matches real-world constraints	Fails later (less immediate feedback) Error messages can be more complex Slightly harder to reason about

Variable validation example

8. Open and review **variables.tf**. Each variable has one or more validation. For this discussion, focus on protocol validation on lines 63-69

```
63   validation {
64     condition = alltrue([
65       for rule_key, rule in var.security_group_rules :
66       contains(["tcp", "udp", "icmp", "-1"], lower(trimspace(rule.proto
67     ])
68     error_message = "Each security group rule protocol must be one of:
69   }
```

9. Test by updating line 30 of **terraform.tfvars** to introduce an invalid protocol...

```
27   security_group_rules = {
28     allow_http_office = {
29       type              = "INGRESS"
30       protocol          = "T CP"
31       destination_ports = ["80"]
32       source_cidrs      = [" 10.0.10.0/24 "]
33       allow_groups      = ["internal_networks"]
34       description       = "allow_http_to_office"
35     }
```

Introduce a space between ‘T’ and ‘CP’ to form an invalid protocol.

10. Plan the deployment and verify that the error is flagged…

```
terraform plan
```

```
Each security group rule protocol must be one of: tcp, udp, icmp, or -1.

This was checked by the validation rule at variables.tf:63,3-13.
```

11. Attempt to apply…

```
terraform apply --auto-approve
```

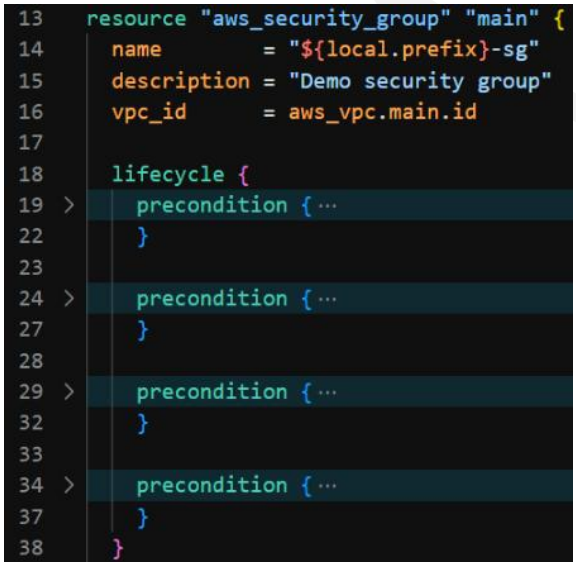
The same error should be returned, **stopping** the deployment. This is a behaviour change from Phase 3 which flagged the error but **allowed** other resources to be deployed.

Reverse the typo and verify that planning completes as expected with fifteen resources proposed…

```
terraform plan
```

lifecycle precondition example

12. Open and review **main.tf**. Lifecycle preconditions have been introduced against the security group using the helper locals introduced into **locals.tf** in Phase 3..



Focusing on one example, lines 19-22 mandate that there must be no unknown allow groups, derived from sanitizing the raw allow groups proposed for each rule in terraform.tfvars and checking they exist in the list of defined allowed groups.

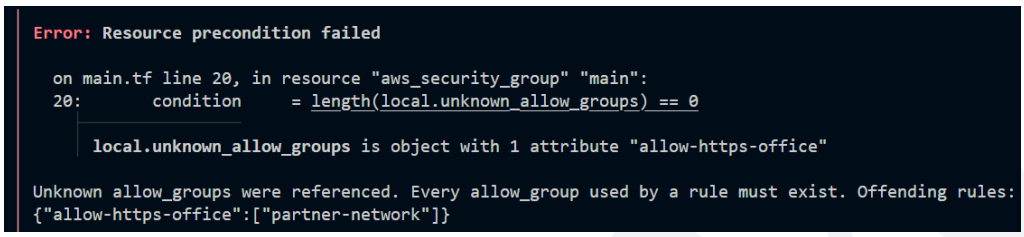
13. Introduce a typo into **terraform.tfvars**;

```
37  Allow-HTTPS-Office = {
38    type              = "INGRESS"
39    protocol          = "Tcp"
40    destination_ports = ["443"]
41    source_cidrs      = ["10.0.10.0/24", "10.0.10.0/24"]
42    allow_groups      = ["internal_networks", "partner_network"]
43    description       = "Allow-HTTPS-to-Office"
44  }
```

On line 42, remove the 's' from 'partner_networks'

14. Plan the deployment and note the early notification...x

    ```
    terraform plan
    ```



    The precondition halts Terraform before resource creation and prevents the configuration from proceeding.

15. Attempt to apply...

    ```
    terraform apply --auto-approve
    ```

    The same error should be returned, **stopping** the deployment. This, again, is a behaviour change from Phase 3, which flagged the error but **allowed** other resources to be deployed.

16. Review and test the other preconditions as time permits.
17. Destroy any deployed resources...

    ```
    terraform destroy --auto-approve
    ```

Lab Conclusions

This lab demonstrates a Terraform data pipeline: Input → Normalize → Detect → Enforce

This pattern is commonly used in production Terraform to ensure that unreliable input is transformed into safe, governed infrastructure.

Each phase builds on the previous one, moving from unreliable input to a fully governed and safe configuration model.

Reflection: If you could only implement one mechanism in a real-world project - variable validation or lifecycle preconditions — which would you choose, and why? What risks would remain?

***** Congratulations, you have completed this lab *****

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026